

SQL Mastery

Day 1 of 15

Databases, Tables & Your First Queries

10

Topics Covered

8

Code Examples

6

Practice Tasks

Today's Goal: By the end of Day 1, you can retrieve data from any table using `SELECT`, filter it with `WHERE`, sort it with `ORDER BY`, and limit results with `LIMIT`.

Table of Contents

1 What is a Database?	2
2 What is SQL?	2
3 Relational Tables — Rows & Columns	2
4 The <code>SELECT</code> Statement	3
5 The <code>WHERE</code> Clause — Filtering Rows	3
6 <code>ORDER BY</code> — Sorting Results	4
7 <code>LIMIT</code> — Restricting Output	4
8 SQL Comments	5
9 Column Aliases with <code>AS</code>	5
10 Putting It All Together	5
★ Practice Tasks	6
✓ Day 1 Cheat Sheet	6

1. What is a Database?

A **database** is an organized collection of data stored electronically. Think of it like a digital filing cabinet — instead of paper files, you have structured tables of data you can search, sort, update, and analyze in milliseconds.

Real-World Example	What the Database Stores
School ERP	Students, attendance, marks, fees, staff
Bank	Accounts, transactions, loans, customers
Amazon	Products, orders, users, reviews, inventory
Hospital	Patients, doctors, prescriptions, appointments

2. What is SQL?

SQL (Structured Query Language) is the language you use to communicate with a relational database. You write SQL commands to ask questions, add records, update them, or delete them. SQL is used by all major databases — PostgreSQL, MySQL, SQLite, Oracle, and MS SQL Server all speak SQL (with minor differences).

Category	Commands	Purpose
DQL	SELECT	Read / query data
DML	INSERT, UPDATE, DELETE	Modify data
DDL	CREATE, ALTER, DROP	Define structure
DCL	GRANT, REVOKE	Permissions
TCL	COMMIT, ROLLBACK	Transactions

■ Today (Day 1) focuses entirely on DQL — specifically SELECT, which is the most used SQL command in the real world. ~80% of your daily SQL will be SELECT queries.

3. Relational Tables — Rows and Columns

In a relational database, data lives in **tables** — like spreadsheets with rows and columns. Each **row** is one record. Each **column** is one attribute. A database usually has many tables that relate to each other.

id	name	age	class	marks
1	Riya Sharma	16	X-A	92
2	Aman Verma	15	IX-B	78
3	Priya Nair	17	XI-A	88
4	Suraj Das	16	X-A	65
5	Meera Joshi	15	IX-B	95

The students table — 5 rows (records), 5 columns (attributes)

4. The SELECT Statement

SELECT retrieves data from one or more tables. It is the most fundamental SQL command.

Syntax

```
SELECT column1, column2, ...  
FROM table_name;
```

Select all columns

```
-- The * means "all columns"  
SELECT * FROM students;
```

Returns every row and every column from the students table

Select specific columns

```
-- Only get name and age (not id, class, marks)  
SELECT name, age FROM students;
```

The order of columns in SELECT controls the order in your output

★ **Tip:** Always prefer selecting specific columns over SELECT * in production code. It is faster, clearer, and avoids breaking when columns are added to the table.

5. The WHERE Clause — Filtering Rows

WHERE lets you filter rows — you get only the rows that match your condition. The database checks each row against the condition and returns only those where it is TRUE.

```
-- Only students aged above 16
SELECT name, age
FROM students
WHERE age > 16;

-- Exact match: only students in class X-A
SELECT name, class
FROM students
WHERE class = 'X-A';

-- Multiple conditions with AND
SELECT name, age, marks
FROM students
WHERE class = 'X-A' AND marks >= 80;
```

WHERE always comes after FROM

Comparison Operators

Operator	Meaning	Example
=	Equal to	WHERE class = 'X-A'
!= or <>	Not equal to	WHERE marks != 0
>	Greater than	WHERE age > 15
<	Less than	WHERE marks < 50
>=	Greater or equal	WHERE salary >= 30000
<=	Less or equal	WHERE age <= 18

6. ORDER BY — Sorting Results

By default SQL returns rows in no guaranteed order. Use ORDER BY to sort — ASC is ascending (A to Z, 1 to 10), DESC is descending (Z to A, 10 to 1).

```
-- Sort students youngest to oldest
SELECT name, age FROM students ORDER BY age ASC;

-- Sort by marks highest first
SELECT name, marks FROM students ORDER BY marks DESC;
```

```
-- Sort by multiple columns: class A-Z, then marks highest

SELECT name, class, marks

FROM students

ORDER BY class ASC, marks DESC;
```

■ ASC is the default — you can omit it. ORDER BY name is the same as ORDER BY name ASC.

7. LIMIT — Restricting Output

LIMIT restricts how many rows are returned. This is crucial for large tables — you never want to accidentally pull 1 million rows.

```
-- Get only the top 5 students by marks

SELECT name, marks

FROM students

ORDER BY marks DESC

LIMIT 5;

-- Skip first 5, get next 5 (pagination)

SELECT name, marks

FROM students

ORDER BY marks DESC

LIMIT 5 OFFSET 5;
```

Database	Syntax
MySQL / PostgreSQL / SQLite	SELECT ... LIMIT 10
SQL Server	SELECT TOP 10 ...
Oracle	WHERE ROWNUM <= 10

8. SQL Comments

Comments are notes in your SQL that the database completely ignores. Always comment your queries — your future self will thank you.

```
-- This is a single-line comment
```

```
SELECT name, age  -- get only name and age
FROM students
WHERE age > 15;   -- only students above 15

/*
    This is a multi-line comment.
    Useful for explaining complex queries.
*/
```

9. Column Aliases with AS

Aliases rename a column in your output — the actual table is not changed. This is useful to make output headers more readable.

```
SELECT
    name      AS student_name,
    age       AS student_age,
    class     AS 'class section'
FROM students;

-- Output headers will be: student_name | student_age | class section
```

■ If an alias contains spaces, wrap it in single quotes (MySQL) or double quotes (PostgreSQL).

10. Putting It All Together

Here is a complete query that uses every concept from today in a single statement.

```
-- Top 5 active students aged above 15, sorted by marks
SELECT
    name      AS student_name,
    age       AS student_age,
    marks     AS total_marks
FROM students
WHERE status = 'active'
    AND age > 15
```

```
ORDER BY marks DESC  
LIMIT 5;
```

Read this top to bottom: WHAT you want → FROM where → FILTERED by → SORTED by → LIMITED to

Clause	Purpose	Required?
SELECT	Which columns to return	Yes
FROM	Which table to read from	Yes
WHERE	Filter rows by condition	No
ORDER BY	Sort the results	No
LIMIT	Cap number of rows returned	No

★ Practice Tasks — Day 1

Use sqliteonline.com or your own PostgreSQL to practice. First create the table below, then run each query.

```
-- Setup: Create the practice table

CREATE TABLE students (
  id      INTEGER PRIMARY KEY,
  name    TEXT,
  age     INTEGER,
  class   TEXT,
  marks   INTEGER,
  status  TEXT
);

INSERT INTO students VALUES (1, 'Riya Sharma', 16, 'X-A', 92, 'active');
INSERT INTO students VALUES (2, 'Aman Verma', 15, 'IX-B', 78, 'active');
INSERT INTO students VALUES (3, 'Priya Nair', 17, 'XI-A', 88, 'active');
INSERT INTO students VALUES (4, 'Suraj Das', 16, 'X-A', 65, 'inactive');
INSERT INTO students VALUES (5, 'Meera Joshi', 15, 'IX-B', 95, 'active');
INSERT INTO students VALUES (6, 'Karan Singh', 18, 'XII-A', 55, 'active');
INSERT INTO students VALUES (7, 'Ananya Roy', 17, 'XI-A', 72, 'inactive');
INSERT INTO students VALUES (8, 'Rahul Gupta', 16, 'X-A', 83, 'active');
```

Run this setup code first, then try the tasks below

Task 1

Select ALL columns and ALL rows from students.

```
SELECT * FROM students;
```

Task 2

Select only name and class of all students.

```
SELECT name, class FROM students;
```

Task 3

Find all students older than 16.


```
SELECT name, age FROM students WHERE age > 16;
```

**Task
4**

Find all students in class 'X-A'.

```
SELECT * FROM students WHERE class = 'X-A';
```

**Task
5**

List all students sorted by marks, highest first.

```
SELECT name, marks FROM students ORDER BY marks DESC;
```

**Task
6**

Get the top 3 students by marks.

```
SELECT name, marks FROM students ORDER BY marks DESC LIMIT 3;
```

**Task
7**

Find active students with marks ≥ 80 , sorted alphabetically.

```
SELECT name AS student_name, marks  
FROM students  
WHERE status = 'active' AND marks  $\geq$  80  
ORDER BY name ASC;
```

Bonus

Find the youngest student in class X-A.

```
SELECT name, age FROM students WHERE class = 'X-A' ORDER BY age ASC LIMIT 1;
```

✓ Day 1 Cheat Sheet — Quick Reference

What you want to do	SQL syntax
Select all columns	<code>SELECT * FROM table;</code>
Select specific columns	<code>SELECT col1, col2 FROM table;</code>
Filter rows	<code>SELECT ... FROM table WHERE condition;</code>
Equal to	<code>WHERE name = 'value'</code>
Greater than	<code>WHERE age > 15</code>
Sort ascending	<code>ORDER BY column ASC</code>
Sort descending	<code>ORDER BY column DESC</code>
Limit rows	<code>LIMIT 10</code>
Skip + limit (pagination)	<code>LIMIT 10 OFFSET 20</code>
Rename column	<code>SELECT name AS student_name</code>
Single-line comment	<code>-- your comment here</code>
Multi-line comment	<code>/* your comment */</code>
Full query order	<code>SELECT → FROM → WHERE → ORDER BY → LIMIT</code>

Day 1 Complete! You now know how to read data from any table, filter it, sort it, limit results, use aliases, and write comments.

Tomorrow (Day 2): AND/OR/NOT, BETWEEN, IN, LIKE, and IS NULL — powerful filtering for real-world queries.